

06-mcp

18. MCP Architecture

18.1 What MCP Is and Why Helios Uses It

The **Model Context Protocol (MCP)** is an open client-server protocol that lets an LLM-driven agent invoke external capabilities ("tools"), read external state ("resources"), and reuse parameterized instruction templates ("prompts") through a single, schema-typed JSON-RPC 2.0 interface. An MCP **server** advertises a manifest of tools — each with a JSON Schema for inputs — and the MCP **client** (here, the Claude Agent SDK runtime) marshals the model's structured tool calls to the server and returns typed results to the model's context window.

Helios uses MCP because the entire product thesis rests on **grounding over generation**: the LLM must NEVER author raw SQL and must NEVER compute a statistic in free text. Instead, every database access and every numerical operation is forced through a narrow, governed, deterministic tool surface. MCP is the mechanism that makes that boundary *physically enforceable* rather than merely a prompt-time suggestion. The model literally has no tool with which to run arbitrary SQL; it can only compose governed metrics. This converts three abstract principles into wiring:

- **Grounding (anti-hallucination):** `semantic-mcp` is the ONLY path to SQL. The model selects canonical metric/dimension names; the server emits validated SQL. A hallucinated column (`event_params.foo`) cannot survive `build_query` because the metric registry does not reference it.
- **Determinism where it matters:** `stats-mcp` is the ONLY path to math. Mix-shift decomposition, significance tests, and forecasts run in audited `scipy/statsmodels` code, byte-for-byte reproducible, never re-derived by the model.
- **Least privilege:** each server holds exactly the credentials and scope it needs. Only `warehouse-mcp` has a BigQuery client. `semantic-mcp` emits SQL *strings* but cannot execute them. `report-mcp` writes briefs but cannot query the warehouse. A compromised or buggy server has a blast radius bounded by its tool catalog.

18.2 Tool-Boundary Rationale

The five-server split is deliberate. SQL *generation* (`semantic-mcp`) is separated from SQL *execution* (`warehouse-mcp`) so that a query is validated and reconciled against canonical totals before a single byte is scanned, and so the model cannot smuggle hand-written SQL into the executor. Math (`stats-mcp`) is separated from data fetch so statistical methods are version-pinned and unit-tested independent of warehouse state. Experiment design (`experiment-mcp`) and narration (`report-mcp`) are isolated because they have zero data-access needs — they operate on findings already produced. The guardrail chain is: **dry_run before every run_query** (cost/byte budget enforcement), **build_query before every dry_run** (no ungoverned SQL reaches the executor), and **stats-mcp before every quantitative claim** (no model-computed numbers).

18.3 Transport, Authentication, Configuration, Statelessness

Concern	Decision
Transport	Local servers (<code>semantic</code> , <code>stats</code> , <code>experiment</code> , <code>report</code>) run over stdio (subprocess, JSON-RPC framed on stdin/stdout) — lowest latency, no network surface. <code>warehouse-mcp</code> runs over HTTP (streamable) so it can live in a network boundary with the BigQuery service account and be shared across runs.
Authentication	<code>warehouse-mcp</code> : GCP service-account JSON via Application Default Credentials (<code>GOOGLE_APPLICATION_CREDENTIALS</code>), scoped to <code>roles/bigquery.dataViewer</code> +

Concern	Decision
	<code>roles/bigquery.jobUser</code> on the project, read-only on <code>bigquery-public-data</code> . HTTP endpoint guarded by a bearer token (<code>HELIOS_WH_TOKEN</code>). <code>stdio</code> servers inherit no warehouse credentials.
Configuration	Declarative <code>mcp_servers.yaml</code> (see 18.10). Each server takes a config block: byte budget, dataset id, metric-registry path, RNG seed.
Statelessness	All tools are stateless request/response ; no session affinity. The lone exception is <code>report-mcp</code> 's memory store (<code>save_diagnosis</code> / <code>recall_prior</code>), which is an explicit, durable side effect backed by a database — not in-process session state. Statelessness makes runs idempotent and horizontally scalable.
Registration	The Agent SDK loads <code>mcp_servers.yaml</code> , spawns/connects each server, fetches its manifest, and exposes the union of tools to agents — filtered per-agent by an allow-list so the Narrator cannot call <code>run_query</code> (see 18.9).

18.4 warehouse-mcp — Governed Execution (enforces verify-then-trust)

Purpose: the sole holder of a BigQuery client. Executes only SQL that already passed `semantic-mcp` , enforces the byte budget via mandatory dry-run, and reconciles metrics against canonical totals.

Tool	Signature (typed inputs)	Output schema	Error modes	Side effects
<code>list_tables</code>	<code>(dataset: str = "ga4_obfuscated_sample_ecommerce")</code>	<code>{tables: [{name, row_count, size_bytes, shard_min, shard_max}]}</code>	<code>DatasetNotFound</code> , <code>AuthError</code>	none (read)
<code>describe_table</code>	<code>(table: str)</code>	<code>{name, schema: [{name, type, mode}], num_rows}</code>	<code>TableNotFound</code>	none
<code>dry_run</code>	<code>(sql: str)</code>	<code>{valid: bool, total_bytes_processed: int, estimated_cost_usd: float, referenced_tables: [str]}</code>	<code>SqlSyntaxError</code> , <code>SchemaError</code>	none (dry-run job)
<code>run_query</code>	<code>(sql: str, max_bytes_billed: int)</code>	<code>{rows: [obj], row_count, bytes_processed, job_id}</code>	<code>ByteBudgetExceeded</code> , <code>QueryTimeout</code> , <code>NotDryRunFirst</code>	BigQuery job (read-only)
<code>reconcile</code>	<code>(metric: str, grain: str)</code>	<code>{metric, grain, canonical_total: float, source: "warehouse"}</code>	<code>UnknownMetric</code>	none

Guardrail: `run_query` rejects any `sql` whose normalized hash was not seen by a prior `dry_run` in the same run, and hard-caps `max_bytes_billed` at the configured budget (`ByteBudgetExceeded` otherwise), keeping query cost per run under the fixed BigQuery byte budget.

```
// dry_run call
{"tool": "dry_run", "arguments": {"sql": "SELECT COUNT(DISTINCT CONCAT(user_pseudo_id, CAST((SELECT value.int_value FROM UNNEST(event_params) WHERE key='ga_session_id') AS STRING))) AS sessions FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` WHERE _TABLE_SUFFIX BETWEEN '20210101' AND '20210131'"}}

// response
{"valid": true, "total_bytes_processed": 248901376, "estimated_cost_usd": 0.00124, "referenced_tables": ["events_2021*"]}
```

18.5 semantic-mcp — The ONLY Path to SQL (enforces grounding)

Purpose: the anti-hallucination layer. Holds the governed metric registry (canonical SQL definitions for `sessions`, `session_conversion_rate`, `revenue`, `aov`, etc.) and the dimension catalog. Composes validated SQL from canonical names only; refuses unknown names.

Tool	Signature	Output schema	Error modes	Side effects
<code>get_metric</code>	<code>(name: str)</code>	<code>{name, sql_template, grain, type, depends_on: [str]}</code>	<code>UnknownMetric</code>	none
<code>list_dimensions</code>	<code>()</code>	<code>{dimensions: [{name, sql_expr, type}]}</code>	none	none
<code>build_query</code>	<code>(metric: str [str], dims: [str], filters: [{dim, op, value}], window: {start, end})</code>	<code>{sql: str, governed: true, metrics, dims}</code>	<code>UnknownMetric</code> , <code>UnknownDimension</code> , <code>InvalidFilter</code> , <code>InvalidWindow</code>	none

`build_query` is the chokepoint: it interpolates only registry-defined templates and dimension expressions, so emitted SQL can reference only real GA4 columns. Output feeds directly into `warehouse-mcp.dry_run`.

```
// build_query call
{"tool": "build_query", "arguments": {"metric": ["sessions", "purchasing_sessions", "session_conversion_rate"], "dims": ["device_category"], "filters": [], "window": {"start": "20210101", "end": "20210131"}}

// response
{"governed": true, "metrics": ["sessions", "purchasing_sessions", "session_conversion_rate"], "dims": ["device_category"], "sql": "WITH s AS (SELECT device.category AS device_category, TO_HEX(MD5(CONCAT(user_pseudo_id, '-', CAST((SELECT value.int_value FROM UNNEST(event_params) WHERE key='ga_session_id') AS STRING)))) AS session_key, MAX(IF(event_name='purchase',1,0)) AS purchased FROM `bigquery-public-data.ga4_obfuscated_sample_ecommerce.events_*` WHERE _TABLE_SUFFIX BETWEEN '20210101' AND '20210131' GROUP BY 1,2) SELECT device_category, COUNT(DISTINCT session_key) AS sessions, SUM(purchased) AS purchasing_sessions, SAFE_DIVIDE(SUM(purchased),COUNT(DISTINCT session_key)) AS session_conversion_rate FROM s GROUP BY 1";}
```

18.6 stats-mcp — The ONLY Path to Math (enforces determinism)

Purpose: every statistic. Implements the core mix-shift vs rate-change decomposition, anomaly detection, significance tests, and forecasting in deterministic, seeded code. The model passes data arrays in and gets numbers out; it never computes.

Tool	Signature	Output schema	Error modes	Side effects
<code>detect_anomaly</code>	<code>(series: [{t, value}], method: "stl" "ewma" "iqr")</code>	<code>{anomalies: [{t, value, score, direction}]}</code>	<code>InsufficientData</code>	none

Tool	Signature	Output schema	Error modes	Side effects
decompose_change	(metric: str, dim: str, t0: [{seg, w, r}], t1: [{seg, w, r}])	{delta_R, mix_effect, rate_effect, interaction, by_segment: [{seg, mix, rate, interaction}]}	SegmentMismatch, InsufficientData	none
significance_test	(a: {n, x}, b: {n, x}, kind: "proportion" "mean")	{p_value, effect_size, ci_low, ci_high, significant: bool}	ZeroSample	none
forecast	(series: [{t, value}], horizon: int)	{forecast: [{t, yhat, lo, hi}], model}	InsufficientData	none
cohort_retention	(events, cohort_grain, periods)	{matrix: [[float]]}	InsufficientData	none
rfm_segment	(users: [{id, recency, frequency, monetary}])	{segments: [{id, r, f, m, label}]}	none	none

`decompose_change` implements exactly the FOUNDATION formula: $\text{mix_effect} = \sum \Delta w_i \cdot r_i(t_0)$, $\text{rate_effect} = \sum w_i(t_0) \cdot \Delta r_i$, $\text{interaction} = \sum \Delta w_i \cdot \Delta r_i$, separating "traffic composition changed" from "behavior changed" and dissolving Simpson's paradox.

```
// decompose_change call
{"tool": "decompose_change", "arguments": {"metric": "session_conversion_rate", "dim": "device_category",
  "t0": [{"seg": "desktop", "w": 0.40, "r": 0.030}, {"seg": "mobile", "w": 0.60, "r": 0.012}],
  "t1": [{"seg": "desktop", "w": 0.30, "r": 0.030}, {"seg": "mobile", "w": 0.70, "r": 0.012}]}
// response
{"delta_R": -0.0018, "mix_effect": -0.0018, "rate_effect": 0.0, "interaction": 0.0,
  "by_segment": [{"seg": "desktop", "mix": -0.0030, "rate": 0.0, "interaction": 0.0},
    {"seg": "mobile", "mix": 0.0012, "rate": 0.0, "interaction": 0.0}]}
```

This shows a pure **mix-shift**: overall conversion fell entirely because traffic moved toward low-converting mobile, with zero behavior change — a finding the Critic cannot refute as a rate problem.

18.7 experiment-mcp — Statistically-Defensible Backlog

Purpose: turns diagnoses into a prioritized, powered experiment backlog. No data access; consumes baselines from prior tools.

Tool	Signature	Output schema	Error modes	Side effects
power_analysis	(baseline: float, mde: float, alpha: float = 0.05, power: float = 0.8)	{n_per_arm: int, total_n: int}	InvalidRate	none
runtime_estimate	(n: int, traffic: float)	{days: float, weeks: float}	ZeroTraffic	none
design_experiment	(hypothesis: str, metric: str)	{test_card: {hypothesis, primary_metric, mde, n_per_arm,	UnknownMetric	none

Tool	Signature	Output schema	Error modes	Side effects
		runtime_days, guardrail_metrics}}		

```
{ "tool": "power_analysis", "arguments": { "baseline": 0.024, "mde": 0.10, "alpha": 0.05, "power": 0.8 } }
// response
{ "n_per_arm": 58420, "total_n": 116840 }
```

18.8 report-mcp — Narration and Memory

Purpose: renders the executive Decision Brief and persists prior diagnoses (the only stateful server). Cannot query the warehouse.

Tool	Signature	Output schema	Error modes	Side effects
render_brief	(findings: [obj])	{brief_md: str, brief_html: str}	EmptyFindings	none
save_diagnosis	(diagnosis: obj)	{id: str, saved: true}	ValidationError	writes memory store
recall_prior	(metric: str, segment: str)	{prior: [{id, t, summary, action_status}]}	none	reads memory store
export	(format: "pdf" "slack" "md")	{uri: str}	UnsupportedFormat	writes artifact

```
{ "tool": "recall_prior", "arguments": { "metric": "session_conversion_rate", "segment": "device_category=mobile" } }
// response
{ "prior": [ { "id": "dx_20210115_07", "t": "2021-01-15", "summary": "mobile mix-shift, $14.2k at risk", "action_status": "experiment_running" } ] }
```

18.9 Per-Agent Tool Allow-Lists

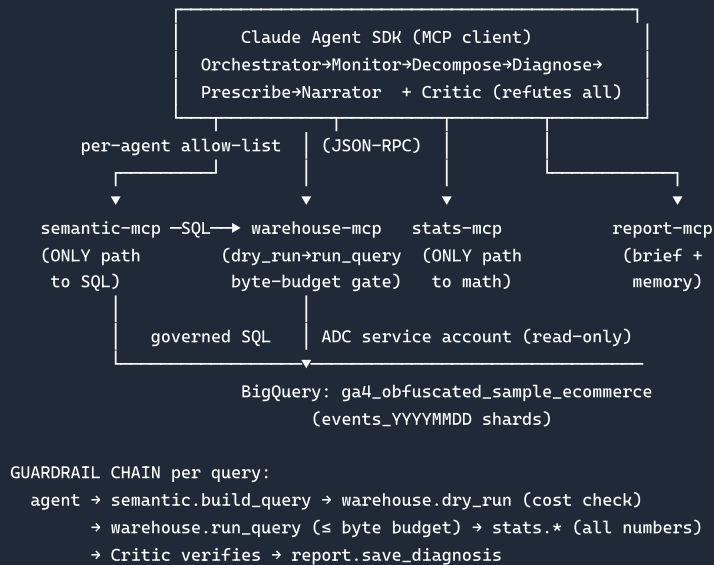
Servers are registered globally, but each of the SEVEN AGENTS sees only the tools it needs, enforcing least privilege at the agent layer:

Agent	Allowed tools
Orchestrator	list_tables, list_dimensions, recall_prior
Monitor	build_query, dry_run, run_query, detect_anomaly
Decompose	build_query, dry_run, run_query, decompose_change
Diagnose	build_query, dry_run, run_query, reconcile, significance_test
Prescribe	power_analysis, runtime_estimate, design_experiment
Narrator	render_brief, export
Critic	reconcile, significance_test, decompose_change, recall_prior

18.10 Configuration File

```
# mcp_servers.yaml
servers:
  warehouse-mcp:
    transport: http
    url: https://helios-wh.internal:8443/mcp
    auth: {type: bearer, token_env: HELIOS_WH_TOKEN}
    config: {dataset: bigquery-public-data.ga4_obfuscated_sample_ecommerce,
             byte_budget: 5368709120, require_dry_run: true}
  semantic-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.semantic"]
    config: {registry: ./semantic/metrics.yaml}
  stats-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.stats"]
    config: {rng_seed: 1729}
  experiment-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.experiment"]
  report-mcp:
    transport: stdio
    command: ["python", "-m", "helios.mcp.report"]
    config: {memory_db: postgres://helios/memory}
```

18.11 Flow Diagram



18.12 Python Tool-Registration Skeleton (warehouse-mcp)

```
# helios/mcp/warehouse.py
import hashlib
from mcp.server.fastmcp import FastMCP
from google.cloud import bigquery

mcp = FastMCP("warehouse-mcp")
_bq = bigquery.Client()           # ADC; least-privilege SA
_BYTE_BUDGET = 5_368_709_120     # fixed per-run cap
_SEEN_DRYRUN = set[str] = set()  # hashes validated this run

def _h(sql: str) → str:
```

```

    return hashlib.sha256(" ".join(sql.split()).lower().encode()).hexdigest()

@mcp.tool()
def dry_run(sql: str) → dict:
    """Validate SQL and estimate scanned bytes WITHOUT executing."""
    cfg = bigquery.QueryJobConfig(dry_run=True, use_query_cache=False)
    job = _bq.query(sql, job_config=cfg)
    _SEEN_DRYRUN.add(_h(sql))
    b = job.total_bytes_processed
    return {"valid": True, "total_bytes_processed": b,
            "estimated_cost_usd": round(b / 1e12 * 6.25, 5),
            "referenced_tables": [t.table_id for t in job.referenced_tables]}

@mcp.tool()
def run_query(sql: str, max_bytes_billed: int) → dict:
    """Execute read-only SQL. Refuses un-dry-run'd or over-budget queries."""
    if _h(sql) not in _SEEN_DRYRUN:
        raise ValueError("NotDryRunFirst: call dry_run before run_query")
    capped = min(max_bytes_billed, _BYTE_BUDGET)
    cfg = bigquery.QueryJobConfig(maximum_bytes_billed=capped)
    job = _bq.query(sql, job_config=cfg) # ByteBudgetExceeded → raises
    rows = [dict(r) for r in job.result()]
    return {"rows": rows, "row_count": len(rows),
            "bytes_processed": job.total_bytes_processed, "job_id": job.job_id}

if __name__ == "__main__":
    mcp.run(transport="streamable-http")

```

This skeleton encodes the load-bearing invariants: `run_query` *cannot* execute SQL that was not first dry-run'd in the same run, and *cannot* exceed the byte budget. Combined with `semantic-mcp` as the sole SQL author and `stats-mcp` as the sole computer of numbers, the architecture guarantees **0 hallucinated columns/metrics (100% governed SQL)** and **100% of findings carrying a significance test and dollar impact** — the FOUNDATION success targets — by construction rather than by convention.